

Face Expression Recognition Using Convolutional Neural Network

Professional Project Documentation

Author: Shafaat Ullah

Environment: Google Colab

Framework: TensorFlow / Keras

Model: Convolutional Neural Network (CNN)

1. Project Overview

Face Expression Recognition is a computer vision and deep learning project that uses a Convolutional Neural Network (CNN) to automatically recognize human facial expressions from images. The model learns visual patterns from facial features and classifies each input image into one of seven expression categories: Angry, Disgust, Fear, Happy, Neutral, Sad, and Surprise.

2. Objectives

The main objectives are to build a CNN-based facial expression recognition system, download and process the dataset using KaggleHub, preprocess facial images, apply data augmentation, train a seven-class CNN, evaluate it using accuracy and classification metrics, analyze errors with a confusion matrix, compare actual and predicted expressions, save the trained model, and predict expressions on new images.

3. Technologies and Libraries

Technology / Library	Purpose
Python	Programming language
Google Colab	Development environment
KaggleHub	Dataset downloading
TensorFlow / Keras	Deep learning and CNN development
NumPy	Numerical operations
Matplotlib	Visualization
Seaborn	Confusion matrix visualization
Scikit-learn	Model evaluation

4. Dataset

The project uses the Face Expression Recognition Dataset available through Kaggle. It is downloaded directly into Google Colab using KaggleHub. Dataset API: [jonathanoheix/face-expression-recognition-dataset](#).

Typical structure:

```
face-expression-recognition-dataset/images/train/{angry, disgust, fear, happy, neutral, sad, surprise}
face-expression-recognition-dataset/images/validation/{angry, disgust, fear, happy, neutral, sad, surprise}
```

5. Data Preprocessing

Images are resized to 48×48 pixels and converted to grayscale, producing an input shape of $48 \times 48 \times 1$. Pixel values are normalized from approximately 0–255 to 0–1. Training augmentation includes rotation, width and height shifting, zoom, and horizontal flipping. Validation images are only normalized so that evaluation remains consistent.

6. Training Configuration

Parameter	Value
Image Size	48×48
Channels	1 (grayscale)
Batch Size	64
Classes	7
Maximum Epochs	10
Optimizer	Adam
Learning Rate	0.0005
Loss	Categorical Crossentropy

7. CNN Architecture

The CNN contains multiple convolutional blocks followed by fully connected layers. The architecture uses 32, 64, 128, and 256 filters, Batch Normalization, Max Pooling, Dropout, Flatten, a Dense layer with 256 neurons, and a final seven-neuron Softmax output layer.

```
Input 48×48×1
↓ Conv2D 32 → Batch Normalization → Conv2D 32 → MaxPooling → Dropout
↓ Conv2D 64 → Batch Normalization → Conv2D 64 → MaxPooling → Dropout
↓ Conv2D 128 → Batch Normalization → Conv2D 128 → MaxPooling → Dropout
↓ Conv2D 256 → Batch Normalization → MaxPooling → Dropout
↓ Flatten → Dense 256 → Batch Normalization → Dropout
↓ Dense 7 → Softmax → Expression Prediction
```

8. CNN Components

Convolutional layers automatically extract facial features such as edges, curves, eyes, eyebrows, and mouth patterns. Batch Normalization stabilizes training. Max Pooling reduces spatial dimensions while preserving important information. Dropout reduces overfitting. ReLU is used in hidden layers, while Softmax converts the final seven outputs into class probabilities.

9. Training and Callbacks

The model is trained for a maximum of **10 epochs** with a batch size of 64. EarlyStopping can stop training before 10 epochs if validation accuracy stops improving. ReduceLROnPlateau reduces the learning rate when validation loss reaches a plateau. ModelCheckpoint saves the best model based on validation accuracy.

10. Model Evaluation

The model is evaluated on the validation dataset using loss and accuracy. A classification report provides precision, recall, F1-score, and support for every expression class. A confusion matrix shows which expressions are correctly classified and which are commonly confused. Actual-versus-predicted images provide an additional visual inspection of model behavior.

11. New Image Prediction

A new image can be uploaded to Google Colab. It is resized to 48×48 , converted to grayscale, normalized, and passed to the trained CNN. The class with the highest Softmax probability is returned as the predicted expression. The displayed probability should be treated as model confidence rather than a guarantee of real-world correctness.

12. Model Saving

The trained model is saved as **face_expression_cnn.keras**. It can later be loaded with TensorFlow/Keras without retraining the network.

13. Project Workflow

Dataset Download → Dataset Exploration → Train/Validation Loading → Image Resizing → Grayscale Conversion → Normalization → Data Augmentation → CNN Construction → Compilation → Training → Validation → Accuracy/Loss Analysis → Classification Report → Confusion Matrix → Actual vs Predicted Analysis → New Image Prediction → Model Saving.

14. Advantages

The CNN automatically learns image features without manual feature engineering, supports multi-class classification, benefits from data augmentation, provides several evaluation methods, and can be saved and reused for future predictions.

15. Limitations

Performance can be affected by visually similar expressions, poor lighting, image quality, face orientation, occlusion, dataset imbalance, and differences between training data and real-world images. Validation accuracy should therefore not be interpreted as guaranteed real-world accuracy in every condition.

16. Future Improvements

Possible improvements include hyperparameter tuning, a larger and more diverse dataset, face detection and alignment before classification, transfer learning with pretrained CNN architectures, improved class balance, stronger controlled augmentation, and testing on unseen real-world images.

17. Conclusion

This project demonstrates a complete CNN-based facial expression recognition pipeline using TensorFlow and Keras. It covers dataset acquisition, preprocessing, augmentation, feature extraction, classification, evaluation, visualization, and prediction. The system recognizes seven facial expression categories and saves the trained model for future reuse.

Project Status: COMPLETED

Author: Shafaat Ullah